

中南大学《数据结构》期中考试试卷

班级：_____ 姓名：_____ 学号：_____

一、 填空题（每空 1 分，共 20 分）

1. 数据的物理结构包括____(1) 顺序____的表示和____(2) 链式____的表示。
2. ____ (3) 树____结构中的数据元素之间存在一个对多个的关系，称为()结构。
3. 一个算法具有 5 个性质：____(4) 有限性____、确定性、可行性，有零个或多个输入、有一个或多个输出。
4. 广义表 $A = (a, b, (c, d), (e, (f, g)))$ ，则 $\text{Head}(\text{Tail}(\text{Head}(\text{Tail}(\text{Tail}(A))))$ 的值为 (d)。
5. 抽象数据类型是指一个逻辑概念上的类型和这个类型上的____(6) 操作集合____。
6. 当线性表的元素总数基本稳定，且很少进行插入和删除操作，但要求以最快的速度存取线性表中的元素时，应采用____(7) 顺序____存储结构。
7. 线性表 $L = (a_1, a_2, \dots, a_n)$ 用数组表示，假定删除表中任一元素的概率相同，则删除一个元素平均需要移动元素的个数是____(8) $(n-1)/2$ ____。
8. 在单链表中设置头结点的作用是____(9) 使得删除插入等操作处理方法相同____。
9. 栈是一种特殊的线性表，允许插入和删除操作的一端称为____(10) 栈顶____。
10. 若一个栈的输入序列是 1, 2, 3, ..., n，输出序列的第一个元素是 n，则第 k 个输出元素是____(11) $n-k+1$ ____。
11. 设有一个 10 阶的对称矩阵 A，采用压缩存储方式，以行序为主存储， a_{11} 为第一元素，其存储地址为 1，每个元素占一个地址空间，则 a_{85} 的地址为 ((12) 33)。
12. 数组 $Q[n]$ 用来表示一个循环队列，f 为当前队列头元素的前一位置，r 为队尾元素的位置，假定队列中元素的个数小于 n，计算队列中元素个数的公式为 ((13) $(n+r-f)\%n$)。
13. 在单链表 L 中，指针 p 所指结点有后继结点的条件是：

- ____(14) p->next!=NULL ____。
14. 串是有 n ($n \geq 0$) 个字符组成的有限序列; 在 C 语言里用 ____ (15) 数组 ____ 存储串, 并在串末尾加上 ____ (16) '\0' ____ 作为串的结束标志。
15. 串的链式存储结构包括 ____ (17) 单字符结点链 ____ 和 ____ (18) 块链 ____。
16. 稀疏矩阵的压缩存储一般采用 ____ (19) 三元组 ____ 顺序表。
17. 递归算法直接或者间接调用了 ____ (20) 函数本身 ____。

二、 综合题 (总分 60)

1. 设计一数据结构, 用来表示某一银行储户的基本信息: 账号、姓名、开户年月日、储蓄类型、存入累加数、利息、帐面总数。【5 分】

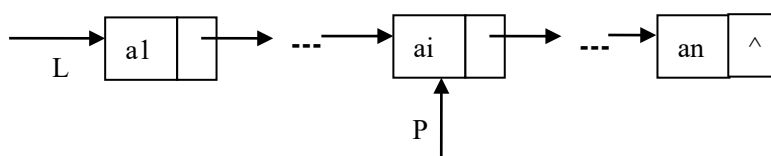
答: struct time

```
{  
  
    int year;  
  
    int mon;  
  
    int day;  
  
};  
  
struct account  
{  
  
    long accountNum;  
  
    char ownerName[20];  
  
    struct time openTime;  
  
    int  accountType;  
  
    double interest;
```

duoble accountSum;

}

2. 已知 L 是带表头结点的非空单链表，且 P 结点既不是第一个元素结点，也不是最后一个元素结点，请写出删除 P 结点的直接后继结点的语句序列：5 分



Q=P->next; (2 分)

P->next=p->next->next; (2 分)

free(Q); (1 分)

3. 设有一个二维数组 A[10][20]，按行存放于一个连续的存储空间中，A[0][0] 的存储地址是 200，每个数组元素占 1 个存储字，则 A[6][2] 的存储字地址是多少。【4 分】 **322**

4. 内存中一片连续空间（不妨假设地址从 1 到 m）提供给两个栈 s1 和 s2 使用，怎样分配这部分存储空间，使得对任一栈，仅当这部分空间全满时才发生上溢。如何判断栈满，栈空，并对两个栈的容量进行分析。（6 分）

答：

把两个栈的栈底分别设定为空间的两头，也就是 1，m。其中一个栈从低地址向高地址增长，另外一个从高地址往低地址存放。这样可以保证空间利用更好。

空、满、容量分析

将 s1,s2 栈底分别设在连续内存空间的两端，栈顶向内存中间进发；

注：栈顶=0，或栈顶=m+1

当|s1.top-s2.top|=1 时，栈满；当一个栈顶为 0，另一个栈顶为 m+1 时，栈空；

容量分析：

从低地址向高地址增长时，容量为栈顶 top 的值；

从高地址往低地址存放时，容量为 m+1-（栈顶 top 的值）。

5. 对比 f(n) 和 g(n)，当 n 接近无穷时，哪个函数增长更快。【4 分】

A $f(n)=(\ln(n!)+5)^2$ $g(n)=13n^{2.5}$
 $g(n)$ 增长速度快

B $f(n)=2^{(n*n*n)}+(2^n)^2$ $g(n)=n^{(n*n)}+n^5$
 $F(n)$ 增长速度快

6. 为什么在单循环链表中设置尾指针比设置头指针更好?【4 分】

答：尾指针是指向终端结点的指针，用它来表示单循环链表可以使得查找链表的开始结点和终端结点都很方便，设一带头结点的单循环链表，其尾指针为 **rear**，则开始结点和终端结点的位置分别是 **rear->next->next** 和 **rear**，查找时间都是 **O(1)**。若用头指针来表示该链表，则查找终端结点的时间为 **O(n)**。

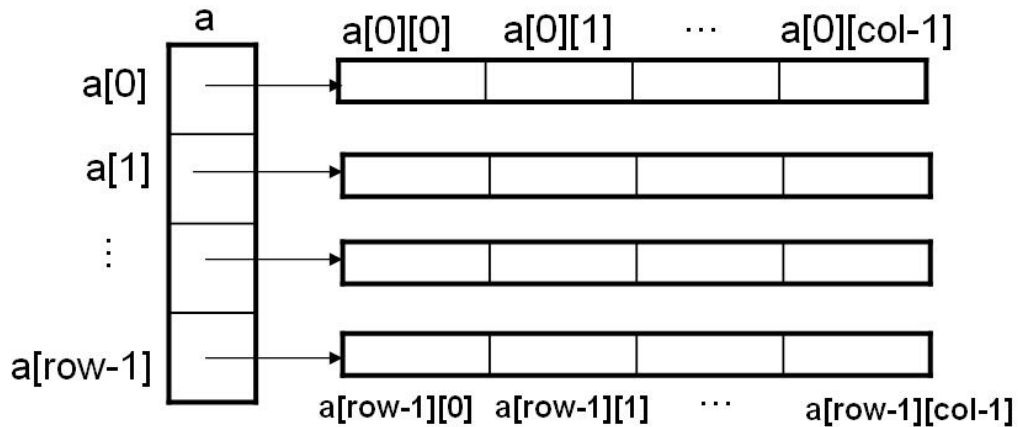
7. 阅读下列程序，该函数什么功能？当 row=4,col=3，请画出执行该函数之后 a 在内存的存储结构【6 分】

```
int **Make2DArray(int row, int col)
{
    int **a, i;

    /*动态申请 row 行个 int 指针类型的内存空间，内存不足时退出*/
    if((a = (int **)malloc(row * sizeof(int *))) == NULL)
    {
        printf("内存空间不够！");
        exit(0);
    }

    for (i = 0; i < row; i++)
        if((a[i] = (int *)malloc(col * sizeof(int))) == NULL)
        {
            printf("内存空间不够！");
            exit(0);
        }
    return a;
}
```

答：动态创建一个 **row x col** 的二维数组；执行该函数之后 **a** 在内存的存储结构：



8. 设有一字符串 $P = "3*y-a/y \uparrow 2"$ ，试写出利用栈将 P 改为 $"3y*ay2 \uparrow /-"$ 的操作步骤。（请用 X 代表扫描该字符串过程中顺序取一字符进栈的操作，用 S 代表从栈中取出一字符加入到新字符串尾的出栈操作。例如，要使 $"ABC"$ 变为 $"BCA"$ ，则操作步骤为 $XXSXS$ ）。【6 分】

XSXXSSXXSXXSXXSSSS

9. 假定 HL 为保存一个集合的有序单链表的表头指针， $item$ 为一个新元素， HL 单链表中的结点包括值域 $data$ 和指向后继结点的指针域 $link$ ，试根据下面算法指出算法功能。（链表结点中保存的数据元素的类型为 $ElemType$ ）【6 分】

功能把 $item$ 插入到有序单链表 HL 中，构建一个新的有序单链表

```
void Unknown(ListNode *HL, ElemType item)
{
    ListNode* newptr= (ListNode *) malloc(sizeof(ListNode));
    newptr->data=item;
    if(HL==NULL || item<HL->data) {
        newptr->link=HL;
        HL=newptr;
        return;
    }
    ListNode *cp, *ap;
    ap=HL; cp=HL->link;
    while(cp!=NULL) {
        if(item<cp->data) break;
        ap=cp;
        cp=cp->link;
    }
}
```

```

newptr->link=cp;
ap->link=newptr;
}

```

10. rear 是指向以循环链表表示的队列的队尾指针，EnLQueue 函数实现把 x 插入到队尾的操作。DeLQueue 函数实现删除队头元素并赋给 x 的操作。根据题意按标号把合适的内容填写到算法后面相应标号的位置。(队列元素的数据类型为 ElemType)【6 分】

```

void EnLQueue(ListNode *rear, ElemType x)
    // rear 是指向以循环链表表示的队列的队尾指针，插入 x 为新的队尾元素。
{
    ListNode *p;
    p=(ListNode *)malloc(sizeof(ListNode));

    p->data=x; __ (1) p->link=rear->link __;

    rear->link=p; rear=p;
};

bool DeLQueue(ListNode *rear, ElemType *x)
    // rear 是指向以循环链表表示的队列的队尾指针，若队列不空，则删除队头元素，
    //并以 x 带回，并返回 true，否则返回 false, x 无意义
{
    if(rear==NULL) return false;
    if (rear->next==rear) {x=rear->data; delete rear; rear=NULL; return
false;}
    ListNode *p=rear->link;
    rear->link=p->link;;

    __ (2) *x=p->data __;

    Free(p);

    __ (3) return true __;

}

```

11.设有下列递归算法：【8 分】

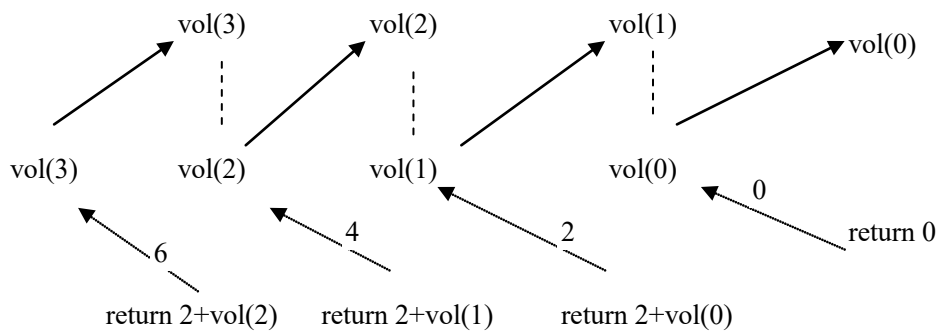
```

int vol(int n)
{
    if(n==0)
        return 0;
    else
        return vol(n-1)+2;
}

```

如该函数被调用时，参数 n 值为 3,函数调用结束时返回值为多少?用图示描述函数的递归调用执行过程。

答：参数 n 值为 3,函数调用结束时返回值为 6，调用执行过程天：



三、 程序设计题（总分 20）

1. 已知带有表头结点的单链表，假设该链表只给出了头指针 L。在不改变链表的前提下，请设计一个尽可能高效的算法，查找链表中倒数第 k 个位置上的结点（k 为正整数）。若查找成功，算法输出该结点的 data 值，并返回 1；否则，只返回 0。要求：

- (1) 描述算法的基本设计思想。
- (2) 根据设计思想，采用 C 程序设计语言描述算法，关键之处请给出简要注释。

(1) 算法基本思想如下：

从头到尾遍历单链表，并用指针 p 指向当前节点的前 k 个节点。当遍历到表的最后一个节点时，指针 p 所指向的节点即为所查找的节点

(2) 详细设计步骤：

增加两个指针变量和一个整型变量，从链表头向后遍历，其中指针 p1 指向当前遍历的节点，指针 p 指向 p1 所指向节点的前 k 个节点，如果 p1 之前没有 k 个节点，那么 p 指向表头节点。

用整型变量 i 表示当前遍历了多少节点，当 $i > k$ 时，指针 p 随着每次遍历也向前移动一个节点。当遍历完成时，p 或者指向表头节点，或者指向链表中倒数第 k 个位置上的节点。

(3) 算法描述：

```
int LocateElemInt(LinkList L, int k)
```

```
{
    p1=L->next;
    p=L;

    i=1;
    while(p1
```

```

    {
        p1=p1->next;
        i++;
        if(i>k)p=p->next;//如果 i>k, 则 p 也往后移
    }
    if(p==L)return 0;//链表中没有 k 个节点
    else
    {
        printf("%d\n", p->data);
        return 1;
    }
}

```

2. 删除栈 S 中所有值为 m 的数据元素。

```

#define STACK_INIT_SIZE    100
#define STACKINCREMENT     10
typedef struct{
    int  *base;
    int  *top;
    int  stacksize;
} Stack;
void Demol( Stack &S, int m)
{ Stack T; int i;
  InitStack (T);//初始化栈
  while (! StackEmpty(S))//判断栈是否为空
    if(( i=Pop(S)) !=m)
        Push( T, i);//入栈操作
  while (! StackEmpty(T))
  {
      i=Pop(T); //出栈操作
      Push(S, i);
  }
}

```

一、单项选择题(共 15 个小题，每小题 1 分，共 15 分)

- 1、以下数据结构中，属于非线性数据结构的是 ()
A、树 B、字符串 C、数组 D、栈
- 2、一个处理二维矩阵的算法的时间复杂度为 $O(n^3)$ ，该算法在利用计算机处理一个 10 列 10 行的矩阵约为 10 个时间单位，则处理 30×30 的矩阵大约需要 () 时间单位，其中 n 为矩阵的阶数。
A、10 B、30 C、300 D、270
- 3、设单链表中指针 p 指向结点 m ，若要删除 m 之后的结点（若存在），则需修改指针的操作为 ()。
A. $p \rightarrow next = p;$ B. $p = p \rightarrow next;$
C. $p = p \rightarrow next \rightarrow next;$ D. $p \rightarrow next = p \rightarrow next \rightarrow next;$
- 4、链表不具有的特点是 ()
A、可随机访问一个元素 B、插入和删除时不需要移动元素
C、不必事先估计访问空间 D、所需空间与线性表的长度成正比
- 5、设依次进入一个栈的元素序列为 d, a, c, b ，则不可能得到的出栈序列为 ()
A、 d, c, b, a B、 a, c, d, b C、 a, b, c, d D、 c, b, d, a
- 6、设栈 S 和队列 Q 初始均为空，若 6 个元素入栈的顺序为 $a_1, a_2, a_3, a_4, a_5, a_6$ 。元素出栈之后立即入队列 Q ，若 6 个元素出队列的顺序为 $a_2, a_4, a_3, a_6, a_5, a_1$ ，则栈 S 的容量至少为 ()
A、4 B、3 C、2 D、5
- 7、循环队列 $A[0 \cdots m-1]$ 存放其元素值，用 $front$ 和 $rear$ 分别表示队头及队尾，则当前队列中的元素个数是 ()
A、 $(rear-front+m)\%m$ B、 $rear-front+1$
C、 $rear-front-1$ D、 $(rear-front)\%m$
- 8、字符串“abcd”中共有()个长度大于 0 的子串
A、11 B、8 C、10 D、12
- 9、设 8 行 10 列的二维数组 $A[1 \cdots 8, 1 \cdots 10]$ 分别以行序为主序和以列序为主序顺序存储时，其首地址相同，那么以以行序为主序时元素 $a[3, 5]$ 的地址与以以列序为主序时()元素的地址相同 (A 无第 0 行第 0 列， $a[i,j]$ 表示第 i 行第 j 列的元素)。
A、 $a[7,3]$ B、 $a[8,3]$ C、 $a[1,4]$ D、 $a[6,5]$
- 10、设广义表 $L = (e, ((f,g), e), (c,d))$ ，则表达式 $Tail(Head(Tail(L)))$ 的值为 ()
A、 (d) B、 (e) C、 g D、 e
- 11、若要求尽可能快地对实数数组进行稳定的排序，则应选 ()
A、快速排序 B、堆排序 C、归并排序 D、基数排序
- 12、设无向图的顶点个数为 n ，则该图最多有 () 条边。
A、 $n-1$ B、 $n(n-1)$ C、 $n(n-1)/2$ D、 n
- 13、采用邻接表存储的图的深度优先遍历算法类似于二叉树的 ()
A、按层遍历 B、后序遍历 C、中序遍历 D、先序遍历
- 14、折半查找有序表 $(4, 6, 10, 12, 20, 30, 50, 70, 88, 99)$ ，若查找元素 60，则它将依次与表中元素 () 比较大小，查找结果是失败。
A、20, 70, 30, 50 B、30, 88, 70
C、20, 50, 70 D、30, 88, 50, 70

15、在对 n 个元素进行冒泡排序的过程中，第一趟排序至多需要进行（ ）对相邻元素之间的交换。

- A、 n B、 $n-1$ C、 $n+1$ D、 $n/2$

二、填空题(1-8 题每空 1 分，第 9 题每空 2 分，共 20 分)

1. 一个算法的时间复杂度为 $(n^3+n^2\log_2n+14n)/n^2$ ，其数量级表示为_____。
2. 假定一棵树的广义表表示为 $A(C, D(E, F, G), H(I, J))$ ，则树中所含的结点数为_____个，树的深度为_____，树的度为_____。
3. 后缀算式 $9\ 2\ 3\ +\ -\ 10\ 2\ /\ -$ 的值为_____。中缀算式 $(3+4X)-2Y/3$ 对应的后缀算式为_____。
4. 若用链表存储一棵二叉树时，每个结点除数据域外，还有指向左孩子和右孩子的两个指针。在这种存储结构中， n 个结点的二叉树共有_____个指针域，其中有_____个指针域是存放了地址，有_____个指针是空指针。
5. AOV 网是一种_____的图。
6. 在一个具有 n 个顶点的无向完全图中，包含有_____条边，在一个具有 n 个顶点的有向完全图中，包含有_____条边。
7. 在堆排序的过程中，对任一分支结点进行筛运算的时间复杂度为_____，整个堆排序过程的时间复杂度为_____。
8. 设线性表中有 n 个数据元素，则在顺序存储结构上实现顺序查找的平均时间复杂度为_____，在链式存储结构上实现顺序查找的平均时间复杂度为_____。
9. 下面程序段的功能是实现二分查找算法，请在下划线处填上正确的语句。

```
struct record{int key; int others;};
int bisearch(struct record r[], int k)
{
    int low=0,mid,high=n-1;
    while(low<=high)
    {
        _____;
        if(r[mid].key==k) return(mid+1); else if(_____) high=mid-1;else low=mid+1;
    }
    return(0);
}
```

三、解答题（5 个小题，共 35 分）

1. （7 分）内存中一片连续的空间（设地址从 1 到 m ），提供给两个栈 s_1 和 s_2 使用，怎样分配这部分存储空间，使得对任一个栈，仅当这部分空间全满时才发生上溢。
2. （7 分）设一棵二叉树的后序遍历序列为 D G J H E B I F C A，中序遍历序列为 D B G E H J A C I F，请完成：

1、画出该二叉树；

- 2、写出该二叉树的先序遍历序列；
- 3、画出该二叉树对应的森林。

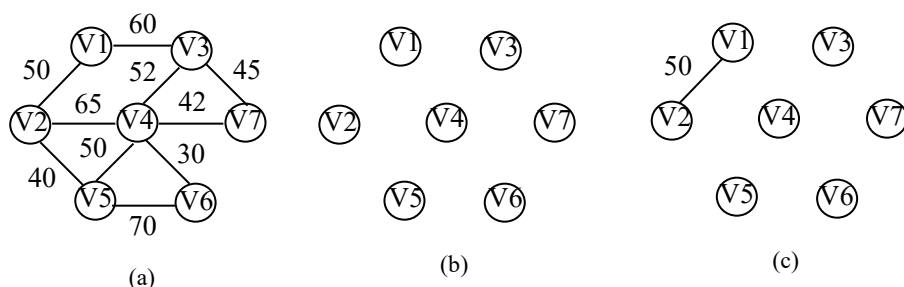
写出算法

3. (7 分) 依次输入表 (38, 52, 25, 74, 68, 16, 30, 54, 90, 72) 中的元素, 生成一棵二叉排序树。

- 1、请画出所得到的二叉排序树；
- 2、若每个元素的查找概率相等, 试计算该二叉排序树的平均查找长度。

4. (7 分) 若用于通信的电文仅由 A-H 八个字母组成, 字母在电文中出现的频率分别为 7, 19, 2, 6, 32, 3, 21, 10。试为这八个字母设计 Huffman 编码 (要求画出对应的 Huffman 树)。

5. (7 分) 已知下图所示的一个网, 按照 Prim 方法, 从顶点 1 出发, 求该网的最小生成树的产生过程。



四、算法设计题 (3 个题, 共 30 分)

1. 设计一个函数修改起泡排序过程以实现双向起泡排序, 即在排序过程中以交替的正、反两个方向进行扫描。若第一趟把关键字最大的记录放在最末尾, 则在第二趟把关键字最小的记录放到最前端, 如此反复行之。
2. 已知非空线性链表由 list 指出, 链结点的构造如下. 请写一算法, 将链表中数据域值最大的那个链结点移到链表的最前面。要求: 不得额外申请新的链结点。

```
struct a{
    int data;
    struct a *link;
};
```

3. 设 A 是包含有 n 个元素的数组, 如果元素 x 在 A 中出现的次数大于 $n/2$, 则称 x 是 A 的主元素。设计一个时间复杂度为 $O(n)$ 的算法, 判断 A 中是否存在主元素。4.

中南大学考试试卷

参考答案与评分标准

2019--2020 学年 2 学期 时间 100 分钟 2020 年 9 月 6 日

数据结构 课程 56 学时 3.5 学分 考试形式: 闭卷

专业年级: 计算机信息类 19 级 总分 100 分, 占总评成绩 60 %

注: 此页不作答题纸, 请将答案写在答题纸上

一、填空题: (每题 1 分, 共 10 分)

1. 某算法时间复杂度 $T(n)$ 表达式是 $100n^4+3n^3+1000n+2\log_2n+999$, 可用大 O 表示成_____。
2. 循环队列用数组 $A[0, \max-1]$ 存放其元素值, 已知其头尾指针分别是 $front$ 和 $rear$, 则该队列满的条件是_____。
3. 设数组 $a[0..10, 2..20]$ 的基地址为 10, 每个元素占 4 个存储单元, 若以行序为主序顺序存储, 则元素 $a[5, 5]$ 的存储地址为_____。
4. 已知一个有向图的邻接矩阵表示, 计算结点 v 的入度的方法是_____。
5. 给定 n 个不同关键字, 则建立的哈夫曼树总结点数为_____。
6. 一个深度为 h 的完全二叉树结点数目最多为_____。
7. 对广义表 $A=((a), (b, (c, d)), ((e), f))$, 运算 $\text{tail}(\text{head}(\text{tail}(\text{head}(\text{tail}(A))))))$ 的结果是_____。
8. 写出带头结点的双向循环链表 L 为空表的条件_____。
9. 表达式 $x*(y/z+a*b)-c$ 的后缀形式是_____。
10. 在长度为 n 的顺序表中的第 i 个元素 ($1 \leq i \leq n$) 之前插入一个元素时, 需向后移动_____个元素。

Solution:

1. $O(n^4)$
2. $(rear+1)\%max==front$
3. 402
4. 顶点 v 所在列中 1 的数目
5. $2n-1$
6. 2^h-1
7. (d)
8. $L->prior==L->next==L$
9. $xyz/ab*+*c-$
10. $n-i+1$

二、选择题: (每题 2 分, 共 20 分)

1. 一个栈的输入序列是 a, b, c, d, e, f , 则栈的不可能输出序列是 ()
A. f, e, d, c, b, a B. a, b, d, c, f, e C. a, c, d, b, f, e D. c, d, a, b, e, f
2. 下列说法正确的是 ()

- A.带权的连通无向图的最小（代价）生成树唯一。
 B.无向图的邻接矩阵一定是对称矩阵，有向图的邻接矩阵一定是非对称矩阵。
 C.拓扑排序可以用来判断有向图有没有环。
 D.深度优先遍历生成树描述了从起点到各顶点的最短路径。
- 3.已知有序表（6, 10, 17, 19, 26, 33, 49, 51, 68），当折半查找值为 19 的元素时，查找成功的比较次数为（ ）。
 A. 1 B. 2 C. 3 D. 4
4. 已知串S='aaab'，则next数组值为（ A ）。
 A. 0123 B. 1123 C. 1231 D. 1211
- 5.给定输入序列{30, 27, 46, 15, 29, 62, 88, 57, 9, 90}构造二叉排序树，则所建二叉排序树的平均查找长度为（ ）。
 A. 3 B. 3.1 C. 3.2 D. 3.3
6. 若广义表 K 满足 $\text{head}(K)=\text{tail}(K)$,则 K 为（ ）。
 A. () B. (()) C. ((), ()) D. ((), (), ())
- 7.在下面的排序方法中，关键字比较的次数与记录的初始排列次序无关的是（ ）。
 A. 快速排序 B. 冒泡排序 C. 插入排序 D. 选择排序
8. 在一棵度为 3 的树中，度为 3 的结点个数为 3，度为 2 的结点个数为 2，则度为 0 的结点个数为（ ）。
 A. 6 B. 7 C. 8 D. 9
- 9.设二叉树的先序遍历序列和后序遍历序列正好相同，则该二叉树满足的条件是（ ）。
 A. 空或只有一个结点 B. 高度等于其结点数
 C. 任一结点无左孩子 D. 任一结点无右孩子
- 10.通过构建最小堆对关键字序列 (49, 56, 69, 32, 28, 77, 89, 18, 46)进行排序，则排序过程中初始堆的序列状态为()
 A. (18, 28, 69, 32, 46, 77, 89, 49, 56) B. (18, 32, 46, 69, 77, 89, 28, 56, 49)
 C. (18, 32, 69, 46, 28, 77, 89, 56, 49) D. (18, 28, 69, 32, 49, 77, 89, 56, 46)

1-5 DCDAB 6-10 BDDAD

三. 计算题（共 5 小题，每小题 8 分，共 40 分）

1. 假设以数组 $\text{seqn}[m]$ 存放循环队列的元素，设变量 rear 和 quelen 分别指示循环队列中队尾元素的位置和元素的个数。

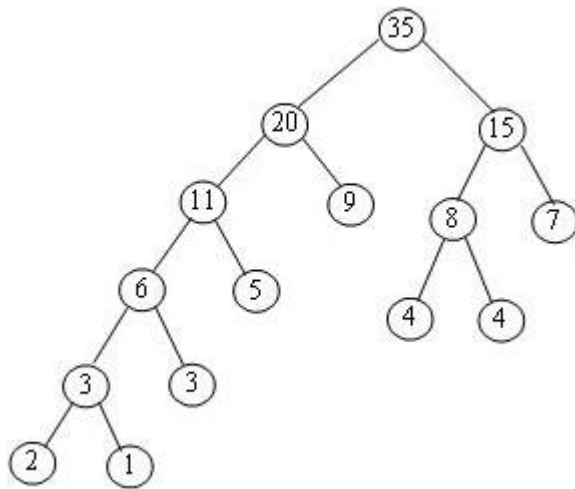
- (1)写出队满的条件表达式；
 (2)写出队空的条件表达式；
 (3)设 $m=40, \text{rear}=13, \text{quelen}=19$,求队头元素的位置；
 (4)写出一般情况下队头元素位置的表达式。

解答：

- (1) $m=\text{quelen}$ (2 分)
 (2) $\text{quelen}=0$ (2 分)
 (3) 队头元素的位置 $\text{front}=34$ (2 分)
 (4) 队头元素的位置 $\text{front}=(\text{rear}+m-\text{quelen})\%m$ (2 分)

2. 已知某字符串 S 中共有 8 种字符，各种字符分别出现 2 次、1 次、4 次、5 次、7 次、3

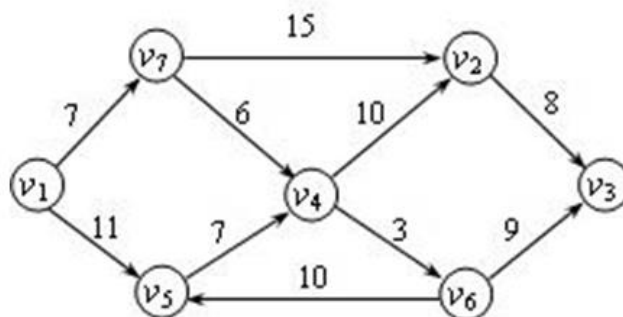
次、4 次和 9 次，对该字符串用[0, 1]进行前缀编码，问该字符串的编码至少有多少位。以
 解答：如图所示，由哈夫曼编码树可得各字母的编码如下：2+6



各字符出现的次数作为叶子结点的权值构造的哈夫曼编码树如图所示。其带权路径长度
 $=2 \times 5 + 1 \times 5 + 3 \times 4 + 5 \times 3 + 9 \times 2 + 4 \times 3 + 4 \times 3 + 7 \times 2 = 98$ ，所以，该字符串的编码长度至少为 98 位。

3. 试对图 3 所示的带权有向图，

- (1) 按 Dijkstra 算法执行过程求从源点 v_1 到其他各顶点的最短路径及最短路径长度。
- (2) Dijkstra 算法在应用中有什么限制？



第 3 题图

解答：(1) 列表求解 5 分，给出路径 2 分，参考答案均见下表。
 (2) 图中含负权边时，算法不一定获得正确结果。1 分

		V1	V2	V3	V4	V5	V6	V7
V1	1	0	∞	∞	∞	11 (V1,V5)	∞	7 (V1,V7)
V7	2	0	22 (V1,V7,V2)	∞	13 (V1,V7,V4)	11 (V1,V5)	∞	7 (V1,V7)
V5	3	0	22 (V1,V7,V2)	∞	13 (V1,V7,V4)	11 (V1,V5)	∞	7 (V1,V7)
V4	4	0	22 (V1,V7,V2)	∞	13 (V1,V7,V4)	11 (V1,V5)	16 (V1,V7,V4,V6)	7 (V1,V7)
V6	5	0	22 (V1,V7,V2)	25 (V1,V7,V4,V6,V3)	13 (V1,V7,V4)	11 (V1,V5)	16 (V1,V7,V4,V6)	7 (V1,V7)
V2	6	0	22 (V1,V7,V2)	25 (V1,V7,V4,V6,V3)	13 (V1,V7,V4)	11 (V1,V5)	16 (V1,V7,V4,V6)	7 (V1,V7)
V3	7	0	22 (V1,V7,V2)	25 (V1,V7,V4,V6,V3)	13 (V1,V7,V4)	11 (V1,V5)	16 (V1,V7,V4,V6)	7 (V1,V7)

4. 设有一组关键字 {9, 7, 26, 16, 37, 22, 89, 56}，采用哈希函数： $H(\text{key}) = (3 \times \text{key}) \bmod 11$ ，表长为 12，用二次探测再散列方法 $H_i = (H(\text{key}) + d_i) \bmod 11$ ($d_i = 1^2, -1^2, 2^2, -2^2, 3^2, \dots$) 解决冲突。

(1) 构造哈希表；(2) 计算查找成功的平均查找长度。

解答：6+2 分

0	1	2	3	4	5	6	7	8	9	10
22	26	37	89	16	9		56			7

平均查找长度： $12/8=3/2$

5. 算法填空题 8 分

如下为二分查找的非递归算法，试将其填写完整。

Int Binsch(ElemType A[], int n, KeyType K)

```

{
    int low=0;
    int high=n-1;
    while (_____)
    {
        int mid=_____
        if (K==A[mid].key) return mid; //查找成功，返回元素的下标
        else if (K<A[mid].key)
            _____; //在左子表上继续查找
        else _____; //在右子表上继续查找
    }
}

```

```

        return -1;        //查找失败，返回-1
    }

```

low<=high ; (low+high)/2 ; high=mid-1 ; low=mid+1 各 2 分

四. 算法设计题（共 2 小题，每题 15 分，共 30 分）酌情给分。

1. 输入一个字符串，内有数字和非数字字符，如：ak123x456 17960?302gef4563，将其中连续的数字作为一个整体，依次存放到一个数组 a 中，如 123 放入 a [0]，456 放入 a [1]，… …。编程统计其共有多少个整数，并输出这些数。

参考算法

```

int CountInt ( )
// 从键盘输入字符串，连续的数字字符算作一个整数，统计其中整数的个数。
{int i=0, a[];        // 整数存储到数组 a, i 记整数个数
  scanf ("%c", &ch); // 从左到右读入字符串
  while (ch!='#')    //'#'是字符串结束标记
  {if (isdigit (ch)) // 是数字字符
    {num=0;          // 数初始化
      while (isdigit (ch) && ch!='#') // 拼数
      {num=num*10+'ch'-'0';
        scanf ("%c", &ch);
      }
      a[i]=num; i++;
      if (ch!='#') scanf ("%c", &ch); // 若拼数中输入了'#'，则不再输入
    } // 结束 while (ch!='#')
    printf ("共有%d 个整数，它们是： "i);
    for (j= 0 ; j<i; j++)
      {printf ("%6d", a[j]);
        if ((j+1) %10==0) printf ("\n"); }// 每 10 个数输出在一行上
  } // 算法结束

```

2. 编写根据无向图 G 的邻接表，判断图 G 是否连通的算法。

将图的深度优先遍历或广度优先遍历算法稍加改造，另设 m 保存访问的顶点个数，若已访问顶点个数 m 小于图中顶点个数 n，则图 G 不连通，否则为连通。现将深度优先遍历的非递归算法改造如下：

采用邻接表表示，类型定义如下：

```

#define MaxV 20
typedef struct node
{int adjvex;
  struct node *next;
} Enode;
typedef struct
{int data;
  Enode *firste;

```

```

    }Vnode;
typedef struct
    {Vnode adjlist[MaxV];
    int n;
    int e;
    }ALGraph;
int connect(ALGraph *G)
{int i,j,Visited[MaxV],m=0,top=0;
  Enode *p,*S[MaxV];
  For(i=0;i<n;i++)
    Visited[i]=0;
  Visited[0]=0;
  Printf("%c",G->adjlist[0].data);
  m++;
  S[top++]=p->next;
  j=p->adjvex;
  if(visited[j]==0)
    {visited[j]=1;
    printf("%c",G->adjlist[j].data);
    m++;
    s[top++]=G->adjlist[j]->firste;
    }
  }
}
if(m<G->a)
  return(0);
else
  return(1);
}

```

中南大学考试试卷

参考答案与评分标准

2020--2021 学年 2 学期 时间 100 分钟 2021 年 7 月 2 日

数据结构 课程 56 学时 3.5 学分 考试形式: 闭卷

专业年级: 计算机信息类 20 级 总分 100 分, 占总评成绩 60 %

注: 此页不作答题纸, 请将答案写在答题纸上

一、填空题: (每题 1 分, 共 10 分)

1. 试确定下列各程序段中前置以记号@的语句的频度表达式为_____。

```
for(i=1; i<=n; i++)
    for(j=1; j<=i; j++)
        for(k=1; k<=j; k++)
            @ x += 1;
```

$$1 + (1+2) + \cdots + (1+2+\cdots+n) \\ = \sum_{i=1}^n \frac{i(i+1)}{2} = \frac{1}{6}n(n+1)(n+2)$$

2. 数组 $Q[n]$ 用来表示一个循环队列, f 为当前队列头元素的前一位置, r 为队尾元素的位置, 假定队列中元素的个数小于 n , 计算队列中元素的公式为 $(n+r-f) \% n$ 。
3. 一个广义表为 $D=(a,b,D)$, 则该广义表的长度为 3。
4. 假设有二维数组 $A_{6 \times 8}$, 每个元素用相邻的 6 个字节存储, 存储器按字节编址, 已知 A 的起始存储位置 (基地址) 为 1000, 按列存储时, 元素 A_{47} 的第一个字节的地址为 1276。
5. $\text{GetHead} [\text{GetTail} [\text{GetHead} [(a, b), (c, d)]]] =$ b 。
6. 设一棵完全二叉树有 700 个结点, 则共有 350 个叶子结点。
7. 用 5 个权值 {3, 2, 4, 5, 1} 构造的哈夫曼 (Huffman) 树的带权路径长度是 33。

8. 设有一稀疏图 G , 则 G 采用 邻接表 存储较省空间。
9. 已知散列表的地址空间为 $A[0..11]$, 散列函数 $H(k) = k \bmod 11$, 采用线性探测法处理冲突。请问在下列数据{25,16,38,47,79,82,51,39,89,151,231}中查找元素 82 时, 需要比较 3 次。
10. 设要将序列 (Q, H, C, Y, P, A, M, S, R, D, F, X) 中的关键码按字母序的升序重新排列, 则: 快速排序一趟扫描的结果是 F H C D P A M Q R S Y X。

二、选择题: (每题 2 分, 共 20 分)

1. 在表长为 n 的顺序表中, 算法的时间复杂度为 $O(1)$ 的操作是 。
- A. 在第 n 个结点之后插入一个结点 B. 在第 i 个结点前插入一个新结点
- C. 删除第 i 个结点 D. 求表长
2. 已知 L 是无表头结点的单链表, 且 P 结点既不是首元结点, 也不是尾元结点, 在 P 结点后插入 S 结点的语句序列是:
- A. $P \rightarrow next = S \rightarrow next; S \rightarrow next = P;$ B. $S \rightarrow next = P \rightarrow next; P \rightarrow next = S;$
- C. $P \rightarrow next = S; S \rightarrow next = P \rightarrow next;$ D. $S \rightarrow next = P; P \rightarrow next = S \rightarrow next;$
3. 双向循环链表的每个结点中包括两个指针 $next$ 和 $previous$, 分别指向该结点的后继和前驱结点。现要删除指针 p 所指向的结点, 下面的操作序列中 是正确的?
- A. $p \rightarrow next \rightarrow previous = p \rightarrow previous; p \rightarrow previous \rightarrow next = p \rightarrow next;$
- B. $p \rightarrow next \rightarrow previous = p \rightarrow next; p \rightarrow previous \rightarrow next = p \rightarrow previous;$
- C. $p \rightarrow previous \rightarrow next = p \rightarrow previous; p \rightarrow next \rightarrow previous = p \rightarrow next;$
- D. $p \rightarrow previous \rightarrow next \rightarrow next = p \rightarrow next; p \rightarrow next \rightarrow previous = p \rightarrow previous;$
4. 若已知一个栈的入栈序列是 $1, 2, 3, \dots, n$, 其输出序列为 $p_1, p_2, p_3, \dots, p_n$, 若 $p_1 = n$, 则 p_i 为:
- A. i B. $n-i$ C. $n-i+1$ D. 不确定
5. 若用一个大小为 6 的数组来实现循环队列, 且当前 $front$ 和 $rear$ 的值分别为 3 和 0, 当

从队列中删除一个元素，再加入两个元素后，front 和 rear 的值分别为_____。

- A. 5 和 1 B. 4 和 2 C. 2 和 4 D. 1 和 5

6. 若 REPLACE (S, S1, S2) 表示用字符串 S2 替换字符串 S 中的子串 S1 的操作，则对于

S="Beijing&Nanjing", S1="Beijing", S2="Shanghai", REPLACE (S, S1, S2) =_____。

- A. "Nanjing & Shanghai" B. "Nanjing & Nanjing"
C. "ShanghaiNanjing" D. "Shanghai & Nanjing"

7. 设哈希表长度为 11，哈希函数为 $H(\text{key}) = \text{key} \bmod 11$ 。表中已有 4 个元素 $H(15) = 4$;
 $H(38) = 5$; $H(61) = 6$; $H(84) = 7$ 其余地址为空，若用二次探测再散列处理冲突，关
键字为 49 的元素的地址是_____。

- A. 3 B. 5 C. 8 D. 9

8. 在一个有向图中，所有顶点的入度之和等于所有顶点的出度之和的 _____ 倍。

- A. 1/2 B. 1 C. 2 D. 4

9. 将 5 个不同的数据进行排序，至多需要比较_____次。

- A. 8 B. 9 C. 10 D. 25

10. 若一组记录的排序码为 (46, 79, 56, 38, 40, 84)，则利用堆排序的方法建立的初始
堆为_____。

- A. 79, 46, 56, 38, 40, 84 B. 84, 79, 56, 38, 40, 46
C. 84, 79, 56, 46, 40, 38 D. 84, 56, 79, 40, 46, 38

三. 计算题 (共 5 小题，每小题 8 分，共 40 分)

1. 利用两个栈 s1,s2 模拟一个队列时，如何用栈的运算实现队列的插入，删除以及判队空

运算。请简述这些运算的算法思想。

解答：

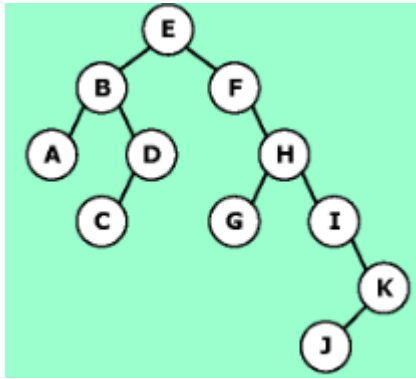
使用两个栈，分别依元素加入的顺序和其反序保存元素，在适当的时机将元素在两个栈中进行转移，从而模拟队列的操作。3 分

令 S1 中元素的顺序为自底向上与元素添加顺序一致，S2 与其相反，则：加入队列时，若 S2 不空，则将 S2 中的元素依次出栈，每出栈一个向 S1 中入栈一个；将入队元素入 S1 栈；从队列中取出时，若 S1 不空，则将 S1 中元素依次出栈，每出栈一个向 S2 中入栈一个；从 S2 栈顶出栈一个即队列中取出的元素。5 分

2. 假设一棵二叉树的先序序列为 EBADCFHGIKJ 和中序序列为 ABCDEFGHIJK。请

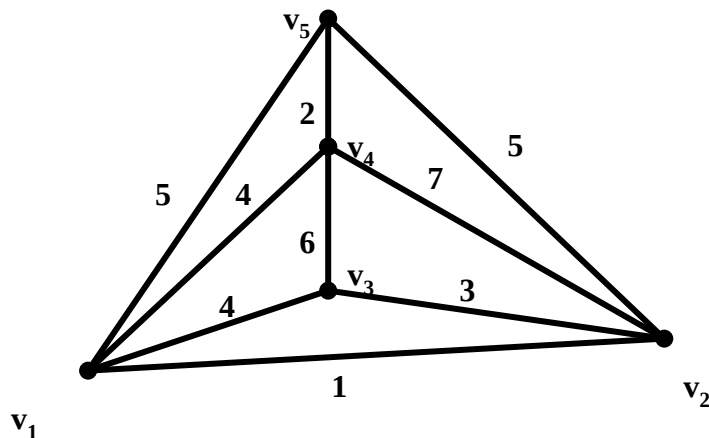
(1) 画出该树。6 分

(2) 在该二叉树对应的森林中有几棵树。2 分



该二叉树对应的森林中有 5 棵树

3. 假设要在某地建造 5 个工厂，拟修筑道路连接这 5 处。经勘探，其道路可按下图的无向边铺设。现在每条边的长度已经测出并标记在图的对应边上，如果我们要求铺设的道路总长度最短，这样既能节省费用，又能缩短工期。请你为该地设计一个方案，并按方案给出每步迭代的结果（假设开始顶点就选为顶点 v_1 ）。



解答：基于图求解最小生成树。3 分

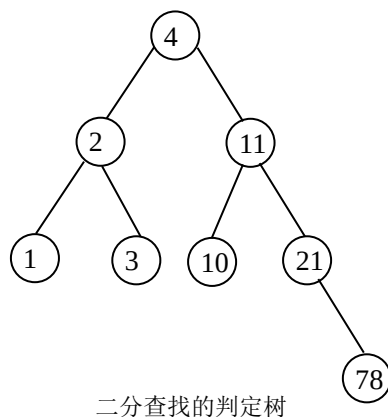
比如使用 Prim 算法求解过程如下：5 分

	V2	V3	V4	V5	U	V-U	k
adiver lowcost	V1 1	V1 4	V1 4	V1 5	{v1}	{v2,v3,v4,v5}	V2
adiver lowcost	0	V2 3	V1 4	V1 5	{v1,v2}	{v3,v4,v5}	V3
adiver lowcost	0	0	V1 4	V1 5	{v1,v2,v3}	{v4,v5}	V4
adiver lowcost	0	0	0	V4 2	{v1,v2,v3,v4}	{v5}	V5
adiver lowcost	0	0	0	0	{V1,V2,V3,V4,V5}	{ }	

4. 给定关键字序列 11, 78, 10, 1, 3, 2, 4, 21, 试分别用二分查找、二叉排序树查找来实现查找, 试画出它们的对应存储形式(二分查找的判定树, 二叉排序树), 并求出每一种查找的成功平均查找长度。

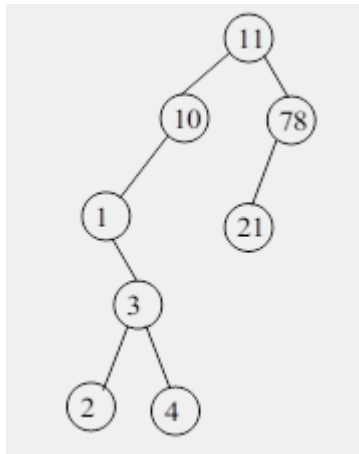
解答:

二分查找的判定树 (中序序列为从小到大排列的有序序列) 3+1 分



二分查找的成功平均查找长度为: $ASL = (1 + 2 \times 2 + 3 \times 4 + 4) / 8 = 2.625$;

二叉排序树 (关键字顺序已确定, 该二叉排序树应唯一) 3+1 分



从图可以得到二叉排序树查找的成功平均查找长度为:

$ASL = (1 + 2 \times 2 + 3 \times 2 + 4 + 5 \times 2) / 8 = 3.125$;

5. 算法填空题 8 分

如下为 Dijkstra 算法的部分代码, 试将其填写完整。

1. `#define SIZE 110`
2. `#define INF 1000000`
3. `int arcs[SIZE][SIZE]; //邻接矩阵存储`

```

4.     int Dist[SIZE];    //dist[i]表示源点到 i 这个点的距离
5.     int visit[SIZE];  //节点是否被访问
6.     int NumOfVertices ;
7.
8.     void Dijkstra(int from ){
9.
10.        int i,j;
11.
12.        for(i = 1 ; i <= NumOfVertices ; i ++){ //初始化
13.            visit[i] = 0;    //一开始每个点都没被访问
14.            Dist[i] = arcs[from][i];    //先假设源点到其他点的距离
15.        }
16.
17.        for(i = 1 ; i < NumOfVertices ; ++i){ //对除源点的每一个点计算
18.            int min = INF; //记录最小Dist[i]
19.            int pos; //记录最小Dist[i] 的点
20.
21.            for(j = 1 ; j <= NumOfVertices ; ++j){
22.                if(!visit[j] && min > Dist[j]){
23.                    pos = j;
24.                    min = Dist[j];
25.                }
26.            }
27.            visit[pos] = 1;
28.
29.            for(j = 1 ; j <= NumOfVertices ; ++j){
30.                if(!visit[j] && (Dist[j] > (Dist[pos] + arcs[pos][j]))){
31.                    Dist[j] = Dist[pos] + arcs[pos][j];
32.                }
33.            }
34.        }
35.
36.    }

```

每空 2 分

四. 算法设计题 (共 3 小题, 每题 10 分, 共 30 分) 酌情给分。

1. 已知指针 ha 和 hb 分别指向两个单链表的头结点, 试写一算法将这两个链表连接在一起 (即令其中一个表的首元结点连在另一个表的最后一个结点之后), 假设指针 hc 指向连接后的链表的头结点, 并要求算法以尽可能短的时间完成连接运算。请分析你的算法和时间

复杂度。

参考代码：

```
void MergeList_L(LinkList &ha, LinkList &hb, LinkList &hc)//合并链表
{
    LinkList pa, pb;
    pa = ha;
    pb = hb;
    while(pa->next && pb->next)
    {
        pa = pa->next;
        pb = pb->next;
    }
    if(!pa->next)
    {
        hc = ha;
        pa->next = hb->next;
        free hb;
    }
    else
    {
        hc = hb;
        pb->next = ha->next;
        free ha;
    }
}
```

8 分

假设两个表长度分别为 m,n, 则时间复杂度： $O(\min(m,n))$ 2 分

2. 假设二叉树采用二叉链存储结构，设计一个算法把一棵含有 **n** 个节点的二叉树 **b** 复制到另一棵二叉树 **t** 中。给出你所设计算法的时间和空间复杂度。

```
typedef struct BTreeNode {
    char data;
    struct BTreeNode *left;
    struct BTreeNode *right;
}BTreeNode;

//由二叉树 BT 复制产生另一棵二叉树 BT1

void copyBTree(BTreeNode*BT, BTreeNode*&BT1) {
    if (BT==NULL) {
        BT1 = NULL;
```

```

    }else{
        BT1 = (BTNode*)malloc(sizeof(BTNode));
        BT1->data = BT->data;
        copyBTree(BT->left, BT1->left);
        copyBTree(BT->right, BT1->right);
    }

```

2 分

8 分

3. 编写算法求出连通图中一个广度优先生成树。

参考代码：类型定义可不写

```

#define    MAXV 100//最大顶点个数

int visited[MAXV];//全局数组

typedef struct ANode
{
    int adjvex;           //该弧的终点位置

    struct ANode *nextarc; //指向下一条弧的指针

    InfoType info;        //可选。该弧的相关信息,这里用于存放权值 n
} ArcNode;//弧的结点结构类型

typedef struct Vnode
{
    int data;             //顶点信息

    ArcNode *firstarc; //指向第一条弧

} VNode;//邻接表头结点的类型

typedef struct
{
    VNode adjlist[MAXV]; //邻接表

    int n,e; //图中顶点数 n 和边数 e

} ALGraph;//图的邻接表类型

void BFS(ALGraph *G, int v)
{

```

```

int queue[MAXV], front=0, rear=0;    //定义循环队列并初始化
int visited[MAXV]; //定义存放结点的访问标志的数组
for (int i=0; i< G->n; i++) visited[i]=0;    //访问标志数组初始化

visited[v]=1; //置已访问标记
queue[rear]=v; //已访问过的顶点 v 进队
rear=(rear+1)%MAXV;
while (front!=rear) //若队列不空时循环
{
    int w=queue[front]; //出队并赋给 w
    front=(front+1)%MAXV;
    ArcNode *p=G->adjlist[w].firstarc; //找与顶点 w 邻接的第一个顶点
    while (p!=NULL)
    {
        if (visited[p->adjvex]==0) //若当前邻接顶点未被访问
        {
            printf("<%d, %d> ", w, p->adjvex); //输出生成树的一条边

            visited[p->adjvex]=1; //置该顶点已被访问的标志
            queue[rear]=p->adjvex; //该顶点 p 的终点进队
            rear=(rear+1)%MAXV;
        }
        p=p->nextarc; //找下一个邻接顶点
    }
}
printf("\n");
}

```